

Imitating Cartpole Policy

Lirui Wang
University of Washington
Seattle, USA
liruiw@uw.edu

Abstract—In this project, we experiment with a cartpole simulation environment to learn a swingup policy from demonstrations. We apply feed-forward network, sequence model, and planning module to imitate an expert policy. Moreover, we experiment with latent encoding of observation for constructing subgoals. We have a few interesting future ideas to experiment with including latent planning and implicit reinforcement learning. Our video can be found at [here](#).

I. INTRODUCTION

In many robotic settings, we can often acquire abundant demonstration data to train a policy from human or other experts while having limitations on environment interactions. For a cartpole swing-up tasks, we assume an existence of an expert that map from state space observation to control that enables a cart to swing up the pole. We start with such a policy and imitate such a policy parametrized by neural network weights and denoted as π_θ that can map from high-dimension pixel observation to control. We also investigate subgoal generation that maps from current observation to possible future states, represented by an autoencoder network represented as f_ϕ . Denote our collected dataset from expert rollout as $p_D = \{(s_t, a_t, s'_t)\}$ containing a state, action, and future state tuple.

II. METHODOLOGY

A. Data Collection

We start with generating data from the given expert swing-up policy. We modify the simulation code to only contain one rollout observation with no extra information. The RGB image is also converted to gray image and crop the cartpole visualization in the center and resize to 128×128 . Then we generate rollouts from the expert policy, i.e state-action pairs. Since the observation needs to encode dynamics, for feedforward model, we concatenate the consecutive 3 images (the one before previous, previous, and current) channel-wise to explicitly provide the model with dynamics information. Our dataset contains 1 million data points for offline training.

B. Behavior Clone

For learning from demonstration, we use a standard supervised learning technique Behavior Clone (BC) that has the following maximum likelihood objective for expert demonstration

$$\max_{\theta} \sum_{s, a \in p_D} \ln(\pi_\theta(a|s))$$



Fig. 1: Behavior Clone Policy

In practice, the log likelihood for gaussian distribution policy with unit covariance matrix reduces the supervised learning to L2 loss. Our network architecture is standard convolutional feature extractor with fully connected regressor. We observe and experiment with the hyperparameters and architecture, denoted as *CNN*. Two problems immediately popped out the diversity of the dataset is not enough and the learned policy can sometimes fail to recover. To that end, we augment the dataset by adding noises to the initial positions and horizontally shifting the image by random pixels. Moreover, BC alone suffers from the distributional shift between the demonstrated states and the policy's own states. [6]. We apply dataset augmentation techniques (Dagger) to interactively incorporate on-policy data by rolling out the current learned policy and label with experts. This policy is denoted as *CNN-Dagger*.

C. Sequence Model for Behavior Clone

Alternatively, one can parameterize the policy with a sequence model such as recurrent neural network (RNN) to implicitly capture the long-termed information. In this case the current observation is one-channel and the information about dynamics and past observations are stored in the hidden states h of the LSTMs [2]. The goal of our model, denoted as *RNN*, is similarly to minimize the distribution mismatch with the expert. Overall, the sequence model with hidden states performs slightly better than the feedforward model which directly considers multiple consecutive frames.

D. Autoencoder for Goal Reconstruction

The goal of the autoencoder (AE), shown in Fig. 2, is to learn a meaningful and minimized encoding-decoding scheme using an iterative optimisation process. It encodes the current observation in latent space with dimension reduction, and reconstruct future goals. The architecture is a standard convolutional encoder-decoder, which takes the current observation and constructs the observation 5-step ahead. Denote our encoder by e and decoder by d , an autoencoder is trained to

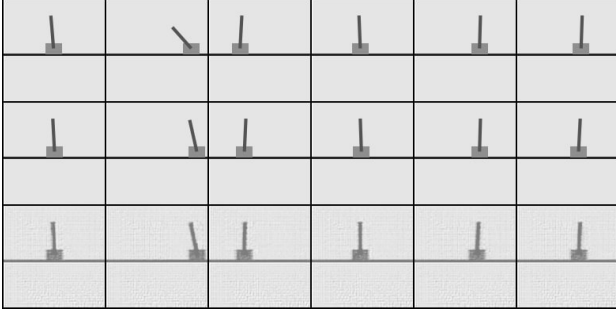


Fig. 2: Autoencoder. The top row is the ground truth future observations and the bottom row is the reconstructed image.



Fig. 3: Conditional Variational AutoEncoder. The first image is the current observation and the second is the future observation, and all the rest are decoded with random samples in standard normal distribution.

minimize $\epsilon(s' - d(e(s)))$ where ϵ denote a binary cross entropy loss since our gray scale images are bounded between 0 and 1. One example is shown in Fig. 2.

E. cVAE for Diverse Subgoal Generation

We also train a conditional Variational AutoEncoder (cVAE) [4] that takes in current time-step observation (not including previous steps) and a latent code to decode potential future images. It shares the same architecture as the autoencoder except that the input is the current observation and future goal. In addition to a reconstruction term, a VAE regularizes the latent space distribution to avoid overfitting and remain smooth. Formally, we approximate the intractable $p(z|s)$ with a Gaussian proxy $q(z|s)$ with mean and variance parametrized by neural networks. Following the variational Bayes analysis [4], we can decompose $\log p(s) = D_{KL}(p(z|s)||q(z|s)) + L(s)$. As the log evidence $\log p(s)$ is fixed with respect to q , maximizing the evidence lower bound $L(s)$ minimizes the KL divergence of $q(z|s)$ from $p(z|s)$, where $L(s)$ can be simplified to $\mathbb{E}_q[\log p(s|z)] - D_{KL}(q(z|s)||p(z))$. Our VAE realization has the form of ELBO $\mathbb{E}_{z \sim q}[-\frac{\|s' - d(z)\|^2}{2}]$, which can be maximized by standard optimization method. In this case we would also condition the decoder on the current observation and tune the balancing ratio of the two terms as in [3]. In practice, we encode the pair in latent space and regularize with a standard Gaussian. For decoding, the current observation is input with the latent code. the KL divergence between two normals has a close form and the reconstruction loss is modified to be the binary cross entropy loss, the same as autoencoder training. From the results shown in 3, the network captures diversity of the datasets.

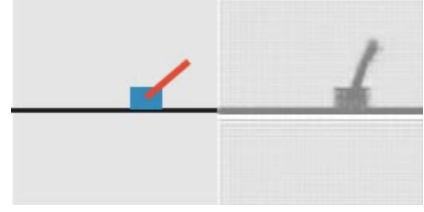


Fig. 4: Goal-Direct Policy



Fig. 5: Goal-Select Policy. Red marker denotes the goal with highest weight. The selected goal latent is a softmax weighted sum over all goals.

F. Goal Directed Policy

Goal information can potentially help reducing the cascaded error and other behavior clone artifacts in imitation learning as well as difficulties caused by sparse rewards in reinforcement learning. This section we combine the sequence model policy and subgoal generative model to verify this hypothesis. The constructed goal in goal-directed policy is fed with the current state into LSTM. *RNN-Goal-Direct* denotes a model that takes in both the features of current observation and goal observation that is generated with the pretrained autoencoder. We also build a novel goal selection mechanism to be used with diverse subgoal generation from VAE, denoted as *RNN-Goal-Select*. The goal-selection mechanism is a simple attention module that takes all reconstructed goals and compute a weights vector. The synthesized goal is computed as a softmax weighted sum over all goal latent. Interestingly, this synthesized goal information sometimes perform much better than the goal-direct policy. We suspect this can be due to a shared information structure or the varying training procedures.

G. Universal Planning Network

Another way to leverage goal information is through planning. [7] uses Universal Planning Network (UPN) to embed differentiable planning within a goal-directed policy. An optimal action plan is generated through gradient descent trajectory optimization. Specifically it unrolls a forward model in a latent space and learn end-to-end to directly optimize a supervised imitation learning objective. Since it involves multiple inner loop optimization steps and large differentiation graph similar to [1], the model training is not intuitively simple. We are actively working on training the UPN to imitate the expert policy.

III. EXPERIMENTS

Conduct experiments to evaluate policy performance requires definition of success for a swing-up tasks. We define one episode success as the angles θ at the final 5 timestep are all less than 0.2 radian away from represents vertical

Behavior Clone Performance					
Metric/Alg.	CNN	CNN-Dagger	RNN	RNN-Goal-Direct	RNN-Goal-Select
Success (Fix)	0.34 ± 0.06	1.00 ± 0.01	0.56 ± 0.04	0.73 ± 0.04	0.85 ± 0.01
Success (Random)	0.28 ± 0.05	0.72 ± 0.03	0.53 ± 0.05	0.57 ± 0.03	0.72 ± 0.04

TABLE I: CNN denotes feedforward model that takes in consecutive frames. CNN-Dagger is augmented with online dataset collection. RNN uses sequence model. RNN-Goal denotes RNN goal-directed policy and RNN-Goal-Select denotes goal selection among goals generated by VAE.

Autoencoder Performance		
Metric/Alg.	AE	cVAE
Test Loss	0.562	0.571

TABLE II

pole angle $\pm\pi$. Overall we run 10 experiments where each experiment contains 100 independent runs. We compute the mean and standard deviation of the results. We also compare the robustness and generalization of the learned network by varying the initial states. The results are shown in the Table I. Being able to interact with the environment online and augmenting dataset turns out to improve the policy. We also see that the information of an goal does improve the policy performance. For the autoencoding performance in Table II, we present the cross entropy loss as the performance criteria. Note that the KL divergence term for VAE is also included since its goal is more than reconstructing the future state. Combining the small divergence with the diverse results, we can see that the VAE captures diversity of the dynamics in its encoding scheme and successfully reconstructs the potential future goals.

IV. CONCLUSION

In conclusion, we have experimented with several standard architecture and methods to learn useful information from an expert policy. In this setting, we qualitatively demonstrate that simple behavior clone with online dataset augmentation can learn a swing-up policy from pixels. Moreover, we show that a useful latent information can be learned to construct the future state or to sample diverse future states. Given a way to encode observations and learn from expert supervision, there are several future directions to experiment with. [5] proposes to learn from suboptimal demonstrations using VAE for goal sampling and goal-directed Behavior Clone. We also experimented with applying [7] to a latent planning framework. In these works, either a goal image is generated from VAE or is given as input. We want to investigate whether a more explicit trajectory optimization in latent space with goal selection mechanism would improve the performance.

REFERENCES

[1] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. In *Proceedings of the 34th International Confer-*

ence on Machine Learning-Volume 70, pages 1126–1135. JMLR. org, 2017.

[2] Felix A Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with lstm. 1999.

[3] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. beta-vae: Learning basic visual concepts with a constrained variational framework. *Iclr*, 2(5):6, 2017.

[4] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[5] Ajay Mandlekar, Fabio Ramos, Byron Boots, Li Fei-Fei, Animesh Garg, and Dieter Fox. Iris: Implicit reinforcement without interaction at scale for learning control from offline robot manipulation data. *arXiv preprint arXiv:1911.05321*, 2019.

[6] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635, 2011.

[7] Aravind Srinivas, Allan Jabri, Pieter Abbeel, Sergey Levine, and Chelsea Finn. Universal planning networks. *arXiv preprint arXiv:1804.00645*, 2018.